VOICE-FIRST · MULTIMODAL · GEMMA

KathaGemma

Adaptive Voice Tutoring System for Bharat

Product Requirements Document (PRD)

Unified specification, architecture, requirements & delivery roadmap

DOCUMENT TYPE	**STATUS**
PRD · Requirements & Roadmap	Hackathon Demo → Production Roadmap
TARGET LEARNERS	**CORE STACK**
Ages 5–7, Class I–II, incl. neurodivergent learners	Gemma + RAG + QLoRA + Multimodal I/O

Prepared for internal reference — architecture, hardware, credentials, datasets, dependencies & phased delivery plan

1. Executive Overview

KathaGemma is a voice-first, multimodal, retrieval-augmented tutoring system built on **Gemma** to support early childhood literacy (ages 5–7, Class I & II level) in India. It is designed for learners who are not well served by text-based AI interfaces — particularly neurodivergent children (e.g. those with ADHD) who struggle with reading focus and do not have a keyboard-driven interaction model.

The system acts as an infinitely patient, highly encouraging tutor that **speaks, hears, and sees**, using structured JSON state frames to dynamically transform the user interface in response to the child's behavior — attention, distraction, mood, and comprehension.

1.1 Core Product Pillars

- **Voice-first interaction** — no text boxes, search bars, or keyboard/settings screens for the child.
- **Multimodal sensing** — speech (Whisper), vision (camera frame tagging), and touch (bubble UI).
- **Structured JSON brain output** — every model turn is forced into a strict schema that directly drives UI state transitions on the client.
- **Curriculum-grounded RAG** — story and vocabulary grounding from NCERT Barkha and Pratham Books StoryWeaver content.
- **Code-switch aware** — fine-tuned to understand and respond in Benglish/Hinglish-style mixed child speech using AI4Bharat's IndicCorpV2.
- **Edge-first future vision** — designed to ultimately run fully offline on low-cost tablets and single-board computers for low-connectivity rural deployment.

1.2 Illustrative End-to-End Use Case — Aarav's Journey

A representative session with a 5-year-old child, **Aarav**, from a village in West Bengal, guided initially by his mother **Sujata**, illustrates the full interaction loop:

Stage	What Happens
1. Initialization	Parent selects a language framework (e.g. Bengali-English/“Benglish”). The AI greets the child by name and narrates the opening scene; the UI shows only a simple illustration — no menus or text fields.
2. Narrating & Reading	The system reads graded story text aloud (speech-to-speech) and prompts the child to respond verbally (e.g. mimic a sound effect). Local Whisper transcribes the response and Gemma reacts with warm, specific encouragement.
3. Distraction Pivot	On detecting prolonged silence and gaze-away via the front camera, the Brain emits a structured JSON frame (e.g. `ui_action: TRIGGER_CAMERA`) that pivots the child into a tactile mini-activity (“find something green”) to recapture attention, then folds the result back into the story.
4. Branching & Comprehension	The story asks a comprehension question and the Brain emits `SHOW_CHOICES`; the UI renders large tactile bubble buttons and the story branches based on the child's tap.

This use case demonstrates the closed loop that the rest of this document formalizes into requirements: sensing → RAG-grounded reasoning → structured JSON → UI/sensor routing → audio playback.

2. System Architecture & Data Flow

KathaGemma divides responsibility across three cooperating modules: **Sensory I/O** (Ears, Eyes, Voice), **The Brain** (Gemma + RAG + LoRA), and the **UI State Manager** on the client.

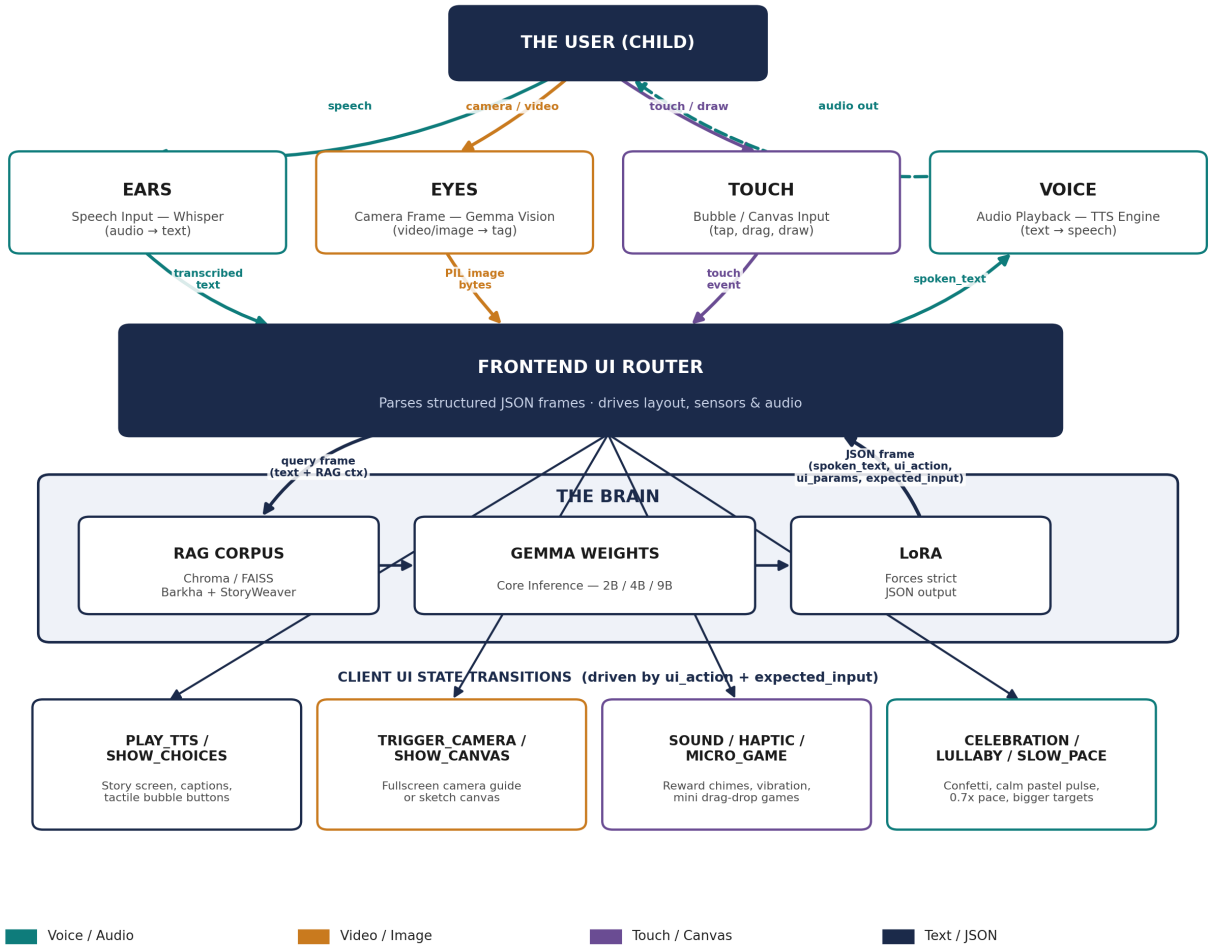

Figure 1 — End-to-end architecture: the child's speech, camera/video, and touch inputs are captured by Ears / Eyes / Touch, routed through the Frontend UI Router, reasoned over by the Brain (RAG + Gemma + LoRA), and returned as a structured JSON frame that drives both spoken audio output (Voice) and the on-screen UI state.

2.1 Module Responsibilities

Module	Responsibility	Key Tech
Ears	Converts child speech to text in real time.	Whisper (local / Whisper.tflite on edge)
Eyes	Captures and tags camera frames for tactile activities and gaze/attention cues.	Gemma Vision encoder

Module	Responsibility	Key Tech
Voice	Renders the model's spoken_text as natural, localized audio.	GCP TTS (cloud) / native Android-iOS TTS, MeloTTS, Bark (edge)
The Brain	RAG-grounded reasoning over curriculum content; LoRA adapter forces strict JSON-schema output encoding the pedagogical state.	Gemma 2B/4B/9B, Chroma/FAISS, QLoRA adapters
UI State Manager	Parses each JSON frame and drives layout transitions, sensor activation, and audio playback on the client.	React / Web / Mobile event-driven router

2.2 High-Level Data Flow

- **Capture** — child speech and/or camera frame is captured by Ears/Eyes.
- **RAG augmentation** — inputs are combined with story/vocabulary context retrieved from the local vector store (Chroma/FAISS) built from Pratham Books and NCERT Barkha content.
- **Brain reasoning** — the augmented prompt is sent to Gemma; QLoRA adapter weights constrain the output to a strict JSON schema.
- **UI routing** — the client parses the JSON frame: extracts `spoken_text` for the TTS pipeline, applies the requested `ui_action` layout transition, and arms the sensor indicated by `expected_input` (speech, image, canvas, or touch).

2.3 Structured JSON Output Contract (Example)

Every Brain turn returns a fixed schema so the client can route UI/sensor state deterministically, without relying on free-text parsing:

```
{
  "spoken_text": "Aarav! I see you have a cool toy! Let's play a game.
    Find something green like a leaf in your room, and show it to
    my camera! Let's help Raju cross the grass!",
  "ui_action": "TRIGGER_CAMERA",
  "ui_params": { "target_prompt": "Find something green" },
  "pedagogical_state": "distraction_recovery",
  "expected_input": "image"
}
```

2.4 UI Actions & Behavioral Archetypes

To keep neurodivergent (ADHD, hyperactive, impatient) and highly sensitive/shy children engaged, the schema supports an extended set of UI commands, each mapped to a behavioral archetype:

UI Action	Target Behavior	Effect
PLAY_TTS	Default storytelling	Standard story screen with illustration + caption text.
TRIGGER_CAMERA	Distracted / gaze-away	Fullscreen camera view with a visual guide/prompt overlay.
SHOW_CANVAS	Creative engagement	Replaces the illustration with a fullscreen sketchpad.
SHOW_CHOICES	Comprehension check	Large tactile bubble buttons built from ui_params.choices.

UI Action	Target Behavior	Effect
PLAY_AUDIO_EFFECT / TRIGGER_SOUNDBOARD	Impatient / distracted	Immediate reward sounds (animal noises, pops, chimes).
TRIGGER_HAPTIC_VIBRATION	Hyperactive / unfocused	Pulses device vibration to draw tactile attention back.
SHOW_MICRO_GAME	Restless / low focus	Short drag-and-drop or bubble-pop game to reset fatigue.
SHOW_CELEBRATION_ANIMATION	Validation-seeking / soft	Fullscreen confetti/star particle overlay on success.
NUDGE_LULLABY / SOFT_PULSE	Over-stimulated / distressed	Pastel breathing-pulse background, camera off, soothing audio.
SLOW_DOWN_PACE	Slow-paced / mild learners	TTS speed reduced to 0.7x; larger touch targets.

3. Fine-Tuning Strategy

3.1 Why Fine-Tune: JSON Layout Alignment

Zero-shot prompting can produce formatting failures — conversational chatter, missing commas, or stray markdown fences — any of which will crash a JSON parser on an edge device. QLoRA (Quantized Low-Rank Adaptation) is applied to permanently bias the adapter weights toward the system prompt and schema, so output is reliably parseable.

3.2 Regional Dialect Code-Switching

Children aged 5–7 rarely speak pure, formal English, Hindi, or Bengali — they naturally mix regional nouns and verbs (Benglish, Hinglish). Standard LLMs trained on formal monolingual corpora either fail to comprehend this input or respond in alienating formal language. **AI4Bharat's IndicCorpV2** is used to adapt the model so it can:

- Parse and understand hybrid speech input (e.g. mapping a regional noun to its English referent).
- Respond using warm, localized vocabulary inside `spoken_text`, mimicking how a mother or local tutor actually talks to the child.

3.3 Training Pipeline (train_lora_gemma.py)

- **Quantization setup** — load Gemma in 4-bit precision via `bitsandbytes` to fit training on low-VRAM budgets (e.g. a free Colab T4).
- **Adapter target modules** — LoRA weights attached to `q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`.
- **Training format** — TRL's `SFTTrainer` formats data as conversational turns pairing a story/child-state context with the target structured JSON reply.

```
<start_of_turn>user
Child is silent. Story context: Raju sees a tree. State: Listening.<end_of_turn>
<start_of_turn>model
{"spoken_text": "Find something green and show it to Raju!",
```

```
"ui_action": "TRIGGER_CAMERA",
"ui_params": {"target_prompt": "Find something green"},
"pedagogical_state": "distraction_recovery",
"expected_input": "image"}<end_of_turn>
```

3.4 Future Target: Fully Offline Mobile Edge Deployment

To serve remote rural schools with poor or expensive connectivity, the long-term vision moves inference entirely on-device:

- **Model quantization** — quantize the fine-tuned model (e.g. Gemma-2-2B) to INT4/INT8, reducing footprint to roughly 1.5–2.2 GB.
- **Execution framework** — deploy via MediaPipe LLM Inference API, Gemma.cpp, or ONNX Runtime Mobile, using the device GPU/NPU.
- **Local sensors** — pair with on-device Whisper.tflite for speech-to-text and native Android/iOS speech engines for zero-latency, zero-cost, private offline tutoring.

4. Detailed Requirements

4.1 Hardware Requirements — Developer Workstation

Required for local inference and QLoRA fine-tuning during development.

Component	Requirement
Operating System	Windows 10/11 or Ubuntu Linux (Linux preferred for GPU operations).
GPU — Minimum	NVIDIA RTX 3060/4060 (8–12 GB VRAM) for 4-bit quantized local execution of Gemma-2-2B.
GPU — Recommended	NVIDIA RTX 3090/4090 (24 GB VRAM) or RTX 4000 Ada (16 GB VRAM) for 8-bit inference and fast local 4-bit QLoRA fine-tuning of Gemma-3-4B-IT / Gemma-2-9B-IT.
GPU — Cloud Fallback	Google Colab Pro (T4, A100 40 GB/80 GB) or Kaggle Notebooks (2x T4, 15 GB VRAM each).
RAM	16 GB minimum, 32 GB recommended.
Storage	50 GB SSD free space for model weights and local vector store indices.

4.2 Hardware Requirements — Target Edge & Mobile Deployment

Target	Specification
Single-board / Edge (future vision)	Raspberry Pi 5 (8 GB) with Coral Edge TPU, Jetson Nano/Orin Nano, or tablets/smartphones via quantized ONNX/TFLite (Gemma-2-2B INT4 using MediaPipe LLM Inference API).
Mobile OS	Android 10+ (API 29+) or iOS 15+.

Target	Specification
Mobile RAM	Minimum 6 GB system RAM (8 GB recommended) to host the quantized model in memory.
On-device inference SDKs	MediaPipe LLM Inference API (quantized Gemma 2B INT4, mobile GPU); Gemma.cpp (lightweight C++ engine, Android NDK-compileable); ONNX Runtime Mobile (INT4/INT8 with NNAPI/CoreML execution providers).
Local speech recognition	Whisper.tflite — base/small/tiny TFLite models (~75–150 MB) with GPU delegate support.
Local text-to-speech	Android native TextToSpeech or iOS AVSpeechSynthesizer, with pre-downloaded locale voice packs (Bengali, Hindi, Indian-accented English) for 0 ms network latency.

4.3 API Credentials & Access Tokens

A `.env` file in the project root must define:

```
# Google AI Studio API Key (core inference during hackathon demo)
GEMINI_API_KEY="your_api_key_here"

# Hugging Face Access Token (downloading Gemma base model weights)
HF_TOKEN="your_huggingface_write_token_here"

# GCP Service Account File (GCP Text-to-Speech API)
GOOGLE_APPLICATION_CREDENTIALS="path/to/gcp-service-account-key.json"
```

4.4 Text-to-Speech Cost Architecture

Google Cloud TTS provides neural-quality, Indian-accented voices. Cost estimation assumes 1 request ≈ 150 characters (25–30 words) at 1 million requests/month (150 million characters/month):

Voice Tier	Price / 1M chars	Billed Volume	Est. Monthly Cost
Standard	$4.00	146M (after 4M free)	$584.00 USD
Neural2 / WaveNet	$16.00	149M (after 1M free)	$2,384.00 USD

Cost Mitigation Strategy:

- **Client-side caching** — cache generated audio (SQLite/IndexedDB on mobile) for static NCERT/StoryWeaver chapter content; replays consume zero additional API requests.
- **Edge hybrid fallback** — use on-device native TTS (Android TextToSpeech, iOS AVFoundation) or lightweight local engines (MeloTTS, quantized Bark) for conversational dialog, reserving GCP TTS for complex pronunciation scaffolding only.

4.5 Dataset Requirements & Sources

Dataset	Purpose	Source
StoryWeaver (Pratham Books)	Knowledge base for story RAG pipeline; graded reading levels for pre-school and beginning readers.	storyweaver.org.in; HF dataset Aunsiels/InfantBooks

Dataset	Purpose	Source
NCERT Barkha Series	Sets vocabulary ceilings and curriculum milestones for Class I & II early literacy.	ncert.nic.in; Internet Archive (barkhaseriesncert)
IndicCorpV2 (AI4Bharat)	Fine-tunes regional dialect code-switching (Benglish, Hinglish) and translation capability.	HF dataset ai4bharat/IndicCorpV2

Loading examples:

```
# StoryWeaver-derived corpus
from datasets import load_dataset
dataset = load_dataset("Aunsiels/InfantBooks", split="train")

# IndicCorpV2 - Bengali subset example
dataset = load_dataset("ai4bharat/IndicCorpV2", "bn", split="train")

# NCERT Barkha PDFs -> text chunks
# Use pypdf or pdfplumber, then load chunks into the vector database.
```

4.6 Software Dependencies

Core Python dependencies grouped by function:

Category	Packages
Core LLM & APIs	google-generativeai>=0.4.0, huggingface_hub>=0.20.0
Vector DB & Document Processing	chromadb>=0.4.22, sentence-transformers>=2.2.2, pypdf>=4.0.0, numpy>=1.24.0
Model Fine-Tuning (QLoRA)	torch>=2.1.2, transformers>=4.38.0, peft>=0.8.2, trl>=0.7.10, bitsandbytes>=0.42.0, accelerate>=0.27.0
Audio & Vision I/O	sounddevice>=0.4.6, scipy>=1.11.4, pyttsx3>=2.90, Pillow>=10.2.0

5. End-to-End Delivery Roadmap

Synthesizing the demo instructions, the fine-tuning plan, and the stated future/edge vision, the project follows a five-phase maturity path from a zero-setup simulation to a fully offline rural deployment.

Phase	Milestone	Scope
Phase 0	Simulation Mode (zero setup)	Run `demo_flow.py` with no API keys; validates the UI state routing logic and behavior-archetype mappings against scripted child scenarios (ADHD distraction, tired child, excitable child).
Phase 1	Active Demo Mode (Gemini API)	Wire in `GEMINI_API_KEY` so live prompts generate real-time structured JSON frames; validates end-to-end schema reliability under zero-shot prompting for the hackathon build.

Phase	Milestone	Scope
Phase 2	RAG Knowledge Grounding	Ingest StoryWeaver/InfantBooks and NCERT Barkha PDFs into Chroma/FAISS; wire retrieval into the prompt pipeline so story content and vocabulary ceilings are curriculum-accurate.
Phase 3	QLoRA Fine-Tuning	Train adapters on Gemma-2-2B / Gemma-3-4B-IT / Gemma-2-9B-IT for (a) strict JSON schema compliance and (b) IndicCorpV2-driven code-switch comprehension (Benglish/Hinglish) using `train_lora_gemma.py` on local GPU or Colab/Kaggle T4/A100.
Phase 4	Voice & Cost Hardening	Integrate GCP TTS for scaffolding-quality pronunciation; implement client-side audio caching and edge-hybrid fallback (MeloTTS/Bark/native OS TTS) to bring per-user cost down before scale testing.
Phase 5	Mobile & Offline Edge Deployment	Quantize the fine-tuned model to INT4/INT8 (~1.5–2.2 GB); deploy via MediaPipe LLM Inference API, Gemma.cpp, or ONNX Runtime Mobile; pair with Whisper.tflite and native OS TTS for a fully offline experience on Android/iOS tablets and Raspberry Pi 5 / Jetson-class edge hardware for low-connectivity rural classrooms.

5.1 Cross-Cutting Study Areas

Beyond the phased milestones, the following areas require ongoing study and validation throughout the roadmap:

- **JSON schema robustness** — stress-testing zero-shot vs fine-tuned reliability, and building a client-side fallback/repair layer for malformed frames on constrained edge hardware.
- **Behavioral detection accuracy** — tuning silence-duration and gaze-tracking thresholds so distraction pivots trigger neither too early (interrupting focus) nor too late (losing the child).
- **Code-switch coverage** — expanding beyond Bengali/Hindi to other IndicCorpV2 languages as the target geography grows.
- **TTS cost curve** — monitoring cache-hit rate in production to validate the Standard vs Neural2/WaveNet cost projections in Section 4.4 against actual usage patterns.
- **On-device performance** — benchmarking INT4 Gemma-2-2B latency and battery draw on the 6–8 GB RAM mobile tier and Raspberry Pi 5 / Jetson Nano-class targets before committing to a default edge SDK (MediaPipe vs Gemma.cpp vs ONNX Runtime Mobile).
- **Safety & content grounding** — ensuring RAG grounding and LoRA fine-tuning keep generated story branches and pedagogical language age-appropriate and curriculum-aligned.

6. Reference: Running the Demo

The reference implementation ships a terminal-based emulator, `demo_flow.py`, that walks through the complete pipeline and UI actions.

Installation

```
pip install -r requirements.txt
```

Simulation Mode (no API keys required)

```
python demo_flow.py
```

Launches an interactive terminal app to select different child-behavior scenarios (ADHD distraction, tired child, excitable child) and observe dynamic UI routing.

Active Mode (Gemini API integration)

```
set GEMINI_API_KEY="your_api_key_here"
python demo_flow.py
```

In Active Mode, the script sends prompts directly to Gemini using the project's system instructions to construct real-time JSON frames.

This document defines the full scope required to build, fine-tune, and progressively deploy KathaGemma from a hackathon demo through to an offline-capable production system.